A Practical Guide to Markov decision process

TOPIC -- MARKOV DECISION PROCESS

-- 01 --

0 = max a (r (t , s , a) + ∂ V (t , s) ∂ s f (t , s , a)) {\displaystyle 0=\max _{a}(r(t,s,a)+{\frac {\partial V(t,s)}{\partial s}}f(t,s,a))}

-- 02 --

Continuous space: Hamilton–Jacobi–Bellman equation In continuous-time MDP, if the state space and action space are continuous, the optimal criterion could be found by solving the Hamilton–Jacobi–Bellman (HJB) partial differential equation. In order to discuss the HJB equation, we need to reformulate our problem

-- 03 --

S {\displaystyle S} is the set of ordered tuples (θ , θ ■ , x , x ■) {\displaystyle (\theta ,{\dot {\theta }},x,{\dot {x}})} given by pole angle, angular velocity, position of the cart and its speed.

-- 04 --

Policy iteration In policy iteration, one first performs Value Determination by solving for V {\displaystyle V} from the linear system described in step one, then performs Policy Improvement by computing π {\displaystyle \pi } as in step two, then repeats both steps until the policy converges. (Policy iteration was invented by Howard to optimize Sears catalogue mailing, which he had been optimizing using value iteration.) Since policy iteration effectively interleaves a linear inverse problem with a nonlinear operation, it may interpreted as a type of relaxation method. This variant has the advantage that there is a definite stopping condition. Since there is a unique solution V {\displaystyle V} for each policy π {\displaystyle \pi } , the algorithm is completed once the Policy Improvement produces the same policy twice consecutively. While there are situations where policy iteration may be faster than value iteration (e.g. when the action space is significantly larger than the state space), policy iteration is usually slower than value iteration for a large number of possible states.

-- 05 --

A {\displaystyle A} is a set of actions called the action space (alternatively, A s {\displaystyle A_{s}} is the set of actions available from state s {\displaystyle s}). As for state, this set may be discrete or continuous.

-- 06 --

max a (t) = π (s (t)) ∫ 0 T r (s (t) , a (t)) d t + D [s (T)] s.t. d s (t) d t = f [t , s (t) , a (t)] {\displaystyle {\begin{aligned}V(s(0),0)={}&\max _{a(t)=\pi (s(t))}\int _{0}^{T}r(s(t),a(t))\,dt+D[s(T)]\\{\text{s.t.}}\quad &{\frac {ds(t)}{dt}}=f[t,s(t),a(t)]\end{aligned}}}

-- 07 --

Category theoretic interpretation Other than the rewards, a Markov decision process (S , A , P) {\displaystyle (S,A,P)} can be understood in terms of Category theory. Namely, let A {\displaystyle {\mathcal {A}}} denote the free monoid with generating set A. Let Dist denote the Kleisli category of the Giry monad. Then a functor A → D i s t {\displaystyle {\mathcal {A}}\to \mathbf {Dist} } encodes both the set S of states and the probability function P. In this way, Markov decision processes could be generalized from monoids (categories with one object) to arbitrary categories. One can call the result (C , F : C → D i s t) {\displaystyle ({\mathcal {C}},F:{\mathcal {C}}\to \mathbf {Dist})} a context-dependent Markov decision process, because moving from one object to another in C {\displaystyle {\mathcal {C}}} changes the set of available actions and the set of possible states.

-- 08 --

R a (s , s ′) {\displaystyle R_{a}(s,s')} is the immediate reward (or expected immediate reward) received after action a {\displaystyle a} is taken to transition from state s {\displaystyle s} to state s ′ {\displaystyle s'} . The reward is in general a random variable. A policy function π {\displaystyle \pi } is a (potentially probabilistic) mapping from state space (S {\displaystyle S}) to action space (A {\displaystyle A}).

-- 09 --

Optimization objective The goal in a Markov decision process is to find a good "policy" for the decision maker: a function π {\displaystyle \pi } that specifies the action π (s) {\displaystyle \pi (s)} that the decision maker will choose when in state s {\displaystyle s} . Once a Markov decision process is combined with a policy in this way, this fixes the action for each state and the resulting combination behaves like a Markov chain (since the action chosen in state s {\displaystyle s} is completely determined by π (s) {\displaystyle \pi (s)}). The

objective is to choose a policy π {\displaystyle \pi } that will maximize some cumulative function of the random rewards, typically the expected discounted sum over a potentially infinite horizon:

-- 10 --

E [∑ t = 0 H − 1 R a t (s t , s t + 1)] {\displaystyle E\left[\sum _{t=0}^{H-1}{R_{a_{t}}(s_{t},s_{t+1})}\right]} (where we choose a t = π (s t) {\displaystyle a_{t}=\pi (s_{t})} , i.e. actions given by the policy). And the expectation is taken over s t + 1 ~ P a t (s t , s t + 1) {\displaystyle s_{t+1}\sim P_{a_{t}}(s_{t},s_{t+1})}

-- 11 --

a set x of possible inputs, a set Φ = { Φ1, ..., Φs } of possible internal states, a set α = { α1, ..., αr } of possible outputs, or actions, with r ≤ s, an initial state probability vector p(0) = ■ p1(0), ..., ps(0) ■, a computable function A which after each time step t generates p(t + 1) from p(t), the current input, and the current state, and a function G: Φ → α which generates the output at each time step. The states of such an automaton correspond to the states of a "discrete-state discrete-parameter Markov process". At each time step t = 0,1,2,3,..., the automaton reads an input from its environment, updates P(t) to P(t + 1) by A, randomly chooses a successor state according to the probabilities P(t + 1) and outputs the corresponding action. The automaton's environment, in turn, reads the action and sends the next input to the automaton.

-- 12 --

Reinforcement learning is an interdisciplinary area of machine learning and optimal control that has, as main objective, finding an approximately optimal policy for MDPs where transition probabilities and rewards are unknown. Reinforcement learning can solve Markov-Decision processes without explicit specification of the transition probabilities which are instead needed to perform policy iteration. In this setting, transition probabilities and rewards must be learned from experience, i.e. by letting an agent interact with the MDP for a given number of steps. Both on a theoretical and on a practical level, effort is put in maximizing the sample efficiency, i.e. minimimizing the number of samples needed to learn a policy whose performance is ε − {\displaystyle \varepsilon -} close to the optimal one (due to the stochastic nature of the process, learning the optimal policy with a finite number of samples is, in general, impossible).